# When the Dike Breaks: Dissecting DNS Defenses During DDoS

Giovane C. M. Moura
SIDN Labs and TU Delft

John Heidemann
USC/Information Sciences Institute

Moritz Müller
SIDN Labs and University of Twente

Ricardo de O. Schmidt
University of Passo Fundo

Marco Davids
SIDN Labs

## ABSTRACT

The Internet's Domain Name System (DNS) is a frequent target of Distributed Denial-of-Service (DDoS) attacks, but such attacks have had very different outcomes—some attacks have disabled major public websites, while the external effects of other attacks have been minimal. While on one hand the DNS protocol is relatively simple, the *system* has many moving parts, with multiple levels of caching and retries and replicated servers. This paper uses controlled experiments to examine how these mechanisms affect DNS resilience and latency, exploring both the client side's DNS *user experience*, and server-side traffic. We find that, for about 30% of clients, caching is not effective. However, when caches are full they allow about half of clients to ride out server outages that last less than cache lifetimes, caching and retries together allow up to half of the clients to tolerate DDoS attacks longer than cache lifetimes, with 90% query loss, and almost all clients to tolerate attacks resulting in 50% packet loss. While clients may get service during an attack, tail-latency increases for clients. For servers, retries during DDoS attacks increase normal traffic up to 8×. Our findings about caching and retries help explain why users see service outages from some real-world DDoS events, but minimal visible effects from others.

## KEYWORDS

DNS, recursive DNS servers, caching, DDoS attacks, authoritative servers

**ACM Reference Format:**
Giovane C. M. Moura, John Heidemann, Moritz Müller, Ricardo de O. Schmidt, and Marco Davids. 2018. When the Dike Breaks:, Dissecting DNS Defenses During DDoS . In *2018 Internet Measurement Conference (IMC '18), October 31-November 2, 2018, Boston, MA, USA.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3278532.3278534

## 1 INTRODUCTION

DDoS attacks have been growing in frequency and intensity for more than a decade. Large attacks have grown from 100 Gb/s in 2012 [4] to over 1 Tb/s in 2017 [31], and 1.7 Tb/s in 2018 [16, 20]. Such attacks are sourced from large botnets (for example, with Mirai peaking at 600k hosts [3]), fueled by the continued deployment of new devices. Gigabit-size attacks are commodities today, selling for a few dollars via DDoS-as-a-Service [41].

The Internet's Domain Name System (DNS) is a popular target of DDoS attacks. DNS is a very visible target, since name resolution is a necessarily step in almost any Internet activity. Root DNS servers have seen multiple attacks over more than a decade [21, 30, 38, 39, 50], as well as threats of attacks [46]. Other authoritative DNS servers have also been attacked, with the huge October 2016 attack against Dyn [12] resulting in disruptions at a number of prominent websites, including Twitter, Netflix and the New York Times [31].

The *outcome* of these attacks on services has varied considerably. The October 2016 Dyn attack is noted for disruption to websites that were using Dyn as their DNS provider, and extortion attempts often include DDoS [32]. However, multiple attacks on the DNS Root have occurred with, as far as has been reported, no visible service outages [38, 39].

An important factor in DNS resilience is heavy use of caching—we believe that differences in use of DNS caching contribute to the very different outcomes when DNS is subject to DDoS attack. Yet understanding DNS caching is difficult, with requests traveling from *stub resolvers* in web browsers and at client computers, to *recursive resolvers* at ISPs, which in turn talk to multiple *authoritative DNS servers.* There are many parts involved to fully resolve a DNS name like www.example.com: while the goal is an IP address (an A or AAAA DNS record), multiple levels of the hierarchy (root, .com, and .example.com) are often on different servers (requiring NS records), and DNSSEC may require additional information (RRSIG, DNSKEY, and DS records). Each of these records may have different cache lifetimes (TTLs), by choice of the operator or because of DNS cache timeouts. We explore caching through controlled experiments (§3) and analysis of real-world use (§4).

Another factor in DNS resilience is recursives that retry queries when they do not receive an answer. Recursives fail to receive answers occasionally due to packet loss, but pervasively during a DDoS attack. We examine how retries interact with caching to mitigate DDoS attacks for loss during DDoS attacks (§5) and their effects on authoritatives (§6).

This paper assesses DNS resilience during DDoS attacks, with the goal of explaining different outcomes from different attacks (§8) through understanding the role of DNS caching, retries, and use of multiple DNS recursive resolvers. It is common knowledge that these factors "help", but knowing *how* and *how much* each contributes builds confidence in defenses. We consider this question both as an operator of an authoritative server, and as a user, defining the *DNS user experience* latency and reliability users should expect.

Publication rights licensed to ACM. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of the United States government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.
*IMC '18, October 31-November 2, 2018, Boston, MA, USA*
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-5619-0/18/10…$15.00
https://doi.org/10.1145/3278532.3278534

Our first contribution is to build an end-to-end understanding of DNS caching. Our key result is that *caching often behaves as expected, but about 30% of the time clients do not benefit from caching.* While prior work has shown DNS resolution infrastructure can be quite complex [45], we establish a baseline DNS user experience by assessing the prevalence of DNS caching in the "wild" through both active measurements (§3) and through analysis of passive data from two DNS zones (.nl and the root zone §4).

Our second contribution is to show that *DNS mechanisms of caching and retries provide significant resilience client user experience during denial-of-service (DDoS) attacks* (§5). For example, about half of the clients continue to receive service during a full outage if caches are filled and do not expire during the attack. Often DDoS attacks cause very high loss, but not a complete outage. When a few queries succeed, caches amplify their benefits, even for attacks that are longer than cache lifetime. With very heavy query loss (90%) on all authoritatives, full caches protect half of the clients, and retries protect 30%. With a DDoS that causes 50% packet loss, nearly all clients succeed, although with greater latency than typical.

Third, we show that there is a large increase in legitimate traffic during DDoS attacks—up to 8× the number of queries (§6). While DNS servers are typically heavily overprovisioned, this result suggests the need to review by how much. It also shows the importance that stub and recursive resolvers follow best practices and exponentially back-off queries after failure so as to not add fuel to the DDoS fire.

Our final contribution is to suggest why users have seen relatively little impact from root servers DDoSes, while customers from some DNS providers quickly felt attacks (§8). When cache lifetimes are longer than the duration of a DDoS attack, many clients will see service for names popular enough to be cached. While many websites use short cache timeouts to support control with DNS-based load balancing, they may wish to consider longer timeouts as part of strategies for DDoS defense. Retries provide additional coverage, preventing failures during large attacks.

All public datasets from this paper is available [22], with our RIPE Atlas data also available from RIPE [35]. Privacy concerns prevent release of .nl and Root data (§4).

## 2 BACKGROUND

As background, we briefly review the components of the DNS ecosystem and how they interact with IP anycast.

### 2.1 DNS Resolvers: Stubs, Recursives, and Authoritatives

Figure 1 shows the relationship between three components of DNS resolvers: stubs and recursive resolvers and authoritative servers. Authoritative servers (authoritatives hereafter) are servers that know the contents of a given DNS zone and can answer queries without asking other servers [9].

Resolvers on the other hand, are servers that can ask, on behalf of others, queries to other servers [18]. *Stub* resolvers run directly on clients and query one or a few *recursive* resolvers (shortened to stubs and recursives here). Recursives perform the full resolution of a domain name, querying one or more authoritatives, while caching responses to avoid repeatedly requesting popular domains (*e.g.*, .com or .google.com). Sometimes recursives operate in multiple

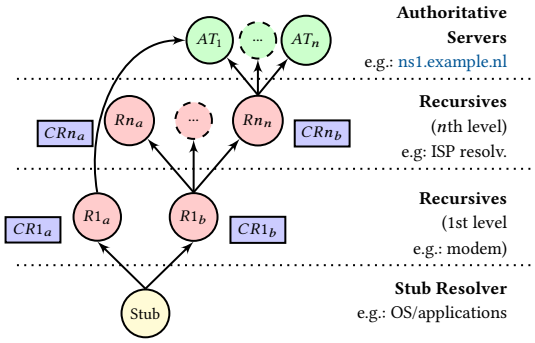



**Figure 1: Relationship between stub resolver (yellow), recursive resolvers (red) with their caches (blue), and authoritative servers (green).**

tiers, with clients talking directly to *R*1 resolvers, that forward queries to other *Rn* resolvers, that ultimately contact authoritatives.

In practice, stubs are part of the client OS or browser, recursives are provided by ISPs, and authoritatives are run by DNS providers or large organizations. Multi-level recursives might have *R*1 at a home router and *Rn* in the ISP, or might occur in large, public DNS providers.

### 2.2 Authoritative Replication and IP Anycast

Replication of a DNS service is important to support high reliability and capacity and to reduce latency. DNS has two complementary mechanisms to replicate service. First, the protocol itself supports *nameserver replication* of DNS service for a zone (.nl or example.nl), where multiple servers operate on different IP addresses, listed by that zone's NS records. Second, each of these servers can run from multiple physical locations with *IP anycast* by announcing the same IP address from each and allowing Internet routing (BGP) to associate clients with each anycast site. Nameserver replication is recommended for all zones, and IP anycast is used by most large zones such as the DNS Root and most top-level domains [21, 40]. IP anycast is also widely used by *public resolvers*, recursive resolvers that are open for use by anyone on the Internet, such as Google Public DNS [10], OpenDNS [26], Quad9 [34], and 1.1.1.1 [1].

### 2.3 DNS Caching with Time-to-Live (TTLs)

DNS depends on caching to reduce latency to users and load on servers. Authoritatives provide responses that are then cached in applications, stub resolvers, and recursive resolvers. We next describe its loose consistency model.

An authoritative resolver defines the lifetime of each result by its *Time-to-Live* (TTL); although TTLs is not usually exposed to users, this information is propagated through recursive resolvers.

Once cached by recursive resolvers, cached results cannot removed; they can only be refreshed response by a new query and response after the TTL expires.

Some recursive resolvers discard long-lived cache entries after a configurable timeout. BIND defaults to dropping entries after 1 week [15], and Unbound after 1 day [25].

Operators select TTLs carefully. Content delivery networks (CDNs) often use DNS to steer users to different content servers. They select

very short TTLs (60 seconds or less) to force clients to re-query frequently, providing opportunities to redirect clients with DNS in response to changes in load or server availability [27]. Alternatively, DNS data for top-level domains often has TTLs of hours or days. Such long TTLs reduce latency for clients (the reply can be reused immediately if it is in the cache of a recursive resolver) and reduce load on servers for commonly used top-level domains and slowly changing DNSSEC information.

# 3 DNS CACHING IN CONTROLLED EXPERIMENTS

To understand the role of caching at recursive resolvers in protection during failure of authoritative servers, we first must understand *how often are cache lifetimes (TTLs) honored.*

In the best-case scenario, authoritative DNS operators may expect clients to be able to reach domains under their zones even if their authoritative servers are unreachable, for as long as cached values in the recursives remain "valid" (*i.e.,* TTL not expired). Given the large variety of recursive implementations, we pose the following question: *from a user point-of-view, can we rely on recursives caching when authoritatives fail?*

To understand cache lifetimes in practice, we carry out controlled measurements from thousands of clients. These measurements determine how well caches work in the field, complementing our understanding of how open source implementations work from their source code. This study is important because operational software can vary and large deployments often use heavily customization or closed source implementations [45].

## 3.1 Potential Impediments to Caching

Although DNS records should logically be cached for the full TTL, a number of factors can shorten cache lifetimes in practice: caches are of limited size, caches may be flushed prematurely, and large resolvers may have fragmented caches. We briefly describe these factors here; understanding how often they occur motivates the measurements we carry out.

Caches are of limited size. Unbound, for example, defaults to a 4 MB limit, but the values are configurable. In practice, DNS results are small enough and caches large enough that cache sizes are usually not a limiting factor. Recursive resolvers may also override record TTLs, imposing either a minimum or maximum value [49].

Caches can be flushed explicitly (at the request of the cache operator), or accidentally on restart of the software or reboot of the machine running the cache.

Finally, some recursive resolvers handle very high request rates—consider a major ISP or public resolver [10, 26, 34]. Large recursive resolvers are often implemented as many separate recursives behind a load balancer or on IP anycast. In such cases the caches may be fragmented with each machine operating an independent cache, or they may share a cache of common names. In practice these may reduce the cache hit rate.

## 3.2 Measurement Design

To evaluate caching we use controlled experiments where we query from specific names to authoritative servers we run from thousands of RIPE Atlas sites. Our goal is to measure whether the TTL we define for the RRs of our controlled domain is honored across recursives.

| TTL | 60 | 1800 | 3600 | 86400 | 3600-10min |
|---|---|---|---|---|---|
| Probes | 9173 | 9216 | 8971 | 9150 | 9189 |
| Probes (val.) | 8725 | 8788 | 8549 | 8750 | 8772 |
| Probes (disc.) | 448 | 428 | 422 | 400 | 417 |
| VPs | 15330 | 15447 | 15052 | 15345 | 15397 |
| Queries | 94856 | 96095 | 93723 | 95780 | 191931 |
| Answers | 90525 | 91795 | 89470 | 91495 | 183388 |
| Answer (val.) | 90079 | 91461 | 89150 | 91172 | 182731 |
| Answers (disc.) | 446 | 334 | 323 | 323 | 657 |

**Table 1: Caching baseline experiments [35].**

**Authoritative servers:** we deploy two authoritatives that answer for our new domain name (cachetest.nl). We place the authoritatives on virtual machines in the same datacenter (Amazon EC2 in Frankfurt, Germany), each at a distinct unicast, IPv4 addresses. Each authoritative runs BIND 9.10.3. Since both authoritatives are in the same datacenter, they will have similar latencies to recursives, so we expect recursives to evenly distribute queries between both authoritative servers [24].

**Vantage Points:** We issue queries to our controlled domain from around 9k RIPE Atlas probes [36]. Atlas Probes are distributed across 3.3k ASes, with about one third hosting multiple *vantage points* (VPs). Atlas software causes each probe to issue queries to each of its local recursive resolvers, so our VPs are the tuple of probe and recursive. The result is that we have more than 15k VPs (Table 1).

**Queries and Caching:** We take several steps to ensure that caching does not interfere with queries. First, each *query* is for a name unique to the probe: each probe requests an AAAA record for {probeid}.cachetest.nl, where {probeid} is the probe's the unique identifier. Each *reply* is also customized. In the AAAA reply we encode three fields that are used to determine the effectiveness of caching (§3.4). Each IPv6 address in the answer is the concatenation of four values (in hex):

- **prefix** is a fixed, 64-bit value (`fd0f:3897:faf7:a375`)
- **serial** is a 8-bit value, incremented every 10 minutes (zone file rotation), allowing us to associate replies with specific query rounds
- **probeid** is the unique Atlas probeID [37] encoded in 8 bits, to associate the query with the reply
- **ttl** is a 16-bit value of the TTL value we configure per experiment

We increment the serial number in each AAAA record and reload the zone (with a new zone serial number), every 10 minutes. The serial number in each reply allows us to distinguish cached results from prior rounds from fresh data in this round.

Atlas DNS queries timeout after 5 seconds, reporting "no answer". We will see this occur in our emulated DDoS events.

We focus on DNS over UDP on IPv4, not TCP or IPv6. We use only IPv4 queries from Atlas Probes, and serve only IPv4 authoritatives, but the IPv6 may be used inside multi-level recursives. Our work could extend to cover other protocols, but we did not want to complicate analysis the orthogonal issue of protocol selection. We focus on DNS over UDP because it is by far the dominant transport protocol today (more than 97% of connections for .nl [47] and most Root DNS servers [14]).

**Query Load**: The query rate of our experiments is designed to explicitly test how queries intersect with TTL experimentation, and not to reproduce real-world traffic rates. Popular domains such as .com will be queried much more frequently than our query rates, so our results represent lower-bounds on caching. In §4 we examine caching rates with real-world names under .nl, testing a range of name popularities.

: **TTL**: TTL values vary significantly in DNS, with top-level domains typically using 1 day TTLs, while CDNs often use short TTLs of 1 or 5 minutes. Given this diversity of configurations, we explicitly design experiments that cover the range from 1 minute to 1 day (60 s and 86400 s TTLs). Thus, rather than trying to capture a single TTL that represents all possible configurations, we study a range of TTLs to explore the full range of caching behavior. §4 examines real-world traffic to provide a view of how well caching works with the distribution of TTLs seen in actual queries.

**Representativeness of Atlas Locations and Software:** It is well known that the global distribution of RIPE Atlas probes is uneven; Europe has far more than elsewhere [5, 6, 43]. Although quantitative data analysis might be generally affected by this distribution bias, our qualitative analysis, contributions and conclusions do not depend on the geographical location of probes.

Atlas probes use identical stub resolver software, but they are deployed in diverse locations (homes, businesses, universities) and so see a diverse set of recursives vendors and versions . Our study therefore represents Atlas "in the wild", and does not try to study specific software versions or vendors. Although we claim our study captures diverse recursive resolvers, we do not claim they are representative of a "typical" Internet client. It complements prior studies on caching by establishing what Atlas sees, an baseline needed when we study DDoS in §5.

## 3.3 Datasets

We carried out five experiments, varying the cache lifetime (TTL) and probing frequency from the VPs. Table 1 lists the parameters of experiments. In the first four measurements, the probing interval was fixed to 20 minutes, and TTL for each AAAA was set to 60, 1800, 3600 and 86400 seconds, all frequently used TTL values. For the fifth measurement we fixed the TTL value to 3600 seconds, and reduced the probing interval to 10 minutes to get better resolution of dynamics.

In each experiment, queries were sent from about 9k Atlas probes. We discard 400–448 of these ("probes (disc.)", about 4.4 to 4.9% of probes) that do not return an answer. Successful Atlas probes query multiple recursive resolvers, each a Vantage Point, so each experiment results in about 15k VPs. We also discard 323–657 answers ("answers (disc.)", about 3.5 to 4.9% of answers) because they report error codes (for example, SERVFAIL and REFUSED [19]), or they are referrals instead of the desired AAAA records [13]. (We provide more detail about referrals in an appendix of our technical report [23].)

Overall, about 93–96k queries to cachetest.nl from the 9k probes at 20 minute pacing, and about double that with 10 minute pacing. Experiments last two to three hours, with no interference between experiments due to use of unique names, We ensure that experiments are isolated from each other. First, we space experiments about one day apart (details in RIPE [35]). Second, the IP addresses (and their records in cachetest.nl) of both authoritative name servers change in each experiment when we restart their VMs. Finally, we change the replies in the AAAA records, so we can detect any stale results (see §3.2).

| TTL | 60 | 1800 | 3600 | 86400 | 3600-10m |
|---|---|---|---|---|---|
| Answers (valid) | 90079 | 91461 | 89150 | 91172 | 182731 |
| 1-answer VPs | 38 | 51 | 49 | 35 | 17 |
| Warm-up (AAi) | 15292 | 15396 | 15003 | 15310 | 15380 |
| Duplicates | 25 | 23 | 25 | 22 | 23 |
| Unique | 15267 | 15373 | 14978 | 15288 | 15357 |
| TTL as zone | 14991 | 15046 | 14703 | 10618 | 15092 |
| TTL altered | 276 | 327 | 275 | 4670 | 265 |
| AA | 74435 | 21574 | 10230 | 681 | 11797 |
| CC | 235 | 29616 | 39472 | 51667 | 107760 |
| CCdec. | 4 | 5 | 1973 | 4045 | 9589 |
| AC | 37 | 24645 | 24091 | 23202 | 47262 |
| TTL as zone | 2 | 24584 | 23649 | 13487 | 43814 |
| TTL altered | 35 | 61 | 442 | 9715 | 3448 |
| CA | 42 | 179 | 305 | 277 | 515 |
| CAdec. | 7 | 3 | 21 | 29 | 65 |

**Table 2: Valid DNS answers (expected/observed)**

## 3.4 TTL distribution: expected vs. observed

We next investigate how often recursive resolvers honor the full TTL provided by authoritative servers. Our goal is to classify the valid DNS answers from Table 1 into four categories, based on where the answer comes from, and where we *expect* it to come from:

- **AA** answers expected and correctly from the authoritative
- **CC** expected and correct from a recursive cache (cache hits)
- **AC** answers from the authoritative, but expected to be from the recursive's cache (a cache miss)
- **CA** answers from a recursive's cache, but expected from the authoritative (an extended cache)

To determine if a query should be answered by the cache of the recursive, we track the state of prior queries and responses, and the estimated TTL. Tracking state is not hard since we know the initial TTL and all queries to the zone, and we encode the serial number and the TTL in the AAAA reply (§3.2).

**Cold Caches and Rewriting TTLs:** We first consider queries made against a cold cache (the first query of a unique name) to test how many recursives *override* the TTL. We know that this happens at some sites, such as at Amazon EC2, where their virtual machines (VMs) default recursive resolver caps all TTLs to 60 s [33].

Table 2 shows the results of our five experiments, in which we classify the valid answers from Table 1. Before classifying them, we first disregard VPs that had only one answer (1-answer VPs) since we cannot evaluate their caches status with one answer only (maximum 51 VPs out of 15,000 for the experiments). Then, we classify the remaining queries as Warm-up queries *AAi*, all of which are type AA (expected and answered by the authoritative server).

We see some duplicate responses; for these we use the timestamp of the very first *AAi* received. We then classify each unique *AAi* by comparing the TTL value returned by the recursive with the expected TTL that is encoded in the AAAA answer (fixed per experiment). The *TTL as zone* line counts the answers we expect to get, while *TTL altered* shows that a few hundred recursive resolvers

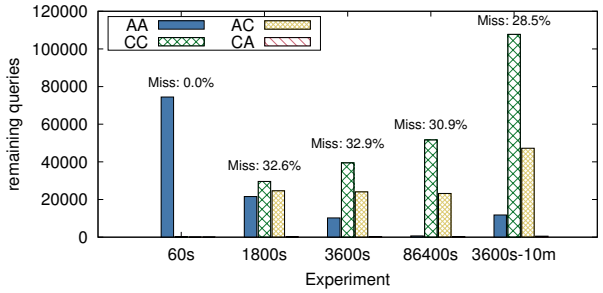



**Figure 2: Classification of subsequent answers with warm cache**

alter the TTL. If these two values differ by more than 10%, we report TTL altered.

We see that the vast majority of recursives honor small TTLs, with only about 2% truncating the TTL (275 to 327 of about 15000, depending on the experiment's TTL). We and others (§7) see TTL truncation from multiple ASes. The exception is for queries with day-long TTLs (86400 s), where 4,670 queries (30%) have shortened TTLs. (Prior work also reported that many public resolvers refreshes at 1 day [48].) We conclude that wholesale TTL shortening does not occur for TTLs of an hour or less.

**TTLs with Warm Cache:** We next consider a warm cache—subsequent queries where we believe the recursive should have the prior answer cached and classify them according to the proposed categories (AA, CC, AC, and CC).

Figure 2 shows a histogram of this classifications (numbers shown on Table 2). We see that most answers we receive show expected caching behavior. For 60 s TTLs (the left bar), we expect no queries to be cached when we re-query 20 minutes (1200 s) later, and we see few cache hits (235 queries – CC row on Table 2 – which are due to TTL rewriting to values larger than 20min.). We see only a handful of CA-type replies, where we expect the authoritative to reply and the recursive does instead. We conclude that under normal operations (with authoritatives responding), recursive resolvers do not serve stale results (as has been proposed when the authoritative cannot be reached [17]).

For longer TTLs we see cache misses (AC responses) fractions of 28 to 33% ($AC$/(Answer_($valid$) – (1−Answers+ Warm-up)). Most of the AC answers did *not* alter the TTL (AC-over), *i.e.,* the cache miss was not due to TTL manipulations (Table 2). We do see 9,715 TTL modifications (about 42% of ACs) when the TTL is 1 day TTLs (86400 s). These TTL truncations are consistent with recursive resolvers that limit cache durations, such as caps of 7 days in BIND [15] and 1 in unbound [25], by default. (We provide more detail about TTL manipulations in an appendix of our technical report [23].)

We conclude that DNS caches are fairly effective, with cache hits about 70% of the time. This estimate is likely a lower bound: we are the only users of our domain, and popular domains would see cache hits due to requests from other users. We only see TTL truncation for day-long TTLs. This result will help us understand the role of caching when authoritatives are under stress.

| TTL | 60 | 1800 | 3600 | 86400 | 3600-10m |
|---|---|---|---|---|---|
| AC Answers | 37 | 24645 | 24091 | 23202 | 47262 |
| Public $R_1$ | 0 | 12000 | 11359 | 10869 | 21955 |
| Google Public $R_1$ | 0 | 9693 | 9026 | 8585 | 17325 |
| other Public $R_1$ | 0 | 2307 | 2333 | 2284 | 4630 |
| Non-Public $R_1$ | 37 | 12645 | 12732 | 12333 | 25307 |
| Google Public $R_n$ | 0 | 1196 | 1091 | 248 | 1708 |
| other $R_n$ | 37 | 11449 | 11641 | 12085 | 23599 |

**Table 3: AC answers public resolver classification.**

## 3.5 Public Recursives and Cache Fragmentation

Although we showed that most requests are cached as expected about 30% are not. We know that many DNS requests are served by public recursive resolvers today, several of which exist [1, 10, 26, 34]. We also know that public recursives often use anycast and load balancing [45] and that that can result in caches that are fragmented (not shared) across many servers. We next examine how many cache misses (type AC replies) are due to public recursives.

Although we control queriers and authoritative servers, there may be multiple levels of recursive resolvers in between. From Figure 1, we see the querier's first-hop recursive (*R1*) and the recursive that queries the authoritative (*Rn*). Fortunately, queries and replies are unique, so we can relate queries to the final recursive knowing the time (the query round) and the query source. For each query $q$, we extract the IP address of $R_n$ and compare against a list of IP addresses for 96 public recursives (given in an appendix of our technical report [23]) we obtain from DuckDuckGo search for "public dns" done on 2018-01-15.

Table 3 reexamines the AC replies from Table 2. With the exception of the measurements with TTL of 60 s, nearly half of AC answers (cache misses) are from queries to public $R_1$ recursives, and about three-quarters of these are from Google's Public DNS. The other half of cache misses start at non-public recursives, but 10% of these eventually emerge from Google's DNS.

Besides identifying public recursives, we also see evidence of cache fragmentation in answers from caches (CC and CA). Sometimes we see serial numbers in consecutive answers decrease. For example, one VP reports serial numbers 1, 3, 3, 7, 3, 3, suggesting that it is querying different recursives, one with serial 3 and another with serial 7 in its cache. We show these occurrences in Table 2 as CCdec. and CAdec. With longer TTLs we see more cache fragmentation, with 4.5% of answers showing fragmentation with day-long TTLs.

From these observations we conclude that cache misses result from several causes: (1) use of load balancers or anycast where servers lack shared caches, (2) first-level recursives that do not cache and have multiple second-level recursives, and (3) caches may reset between the somewhat long probing interval (10 or 20 minutes). Causes (1) and (2) occur in public resolvers (confirmed by Google [10]) and account for about half of the cache misses in our measurements.

# 4 CACHING PRODUCTION ZONES

In §3 we show that about one-third of queries do not conform with caching expectations, based on controlled experiments to our test domain. (Results may be better for caches that prioritize popular

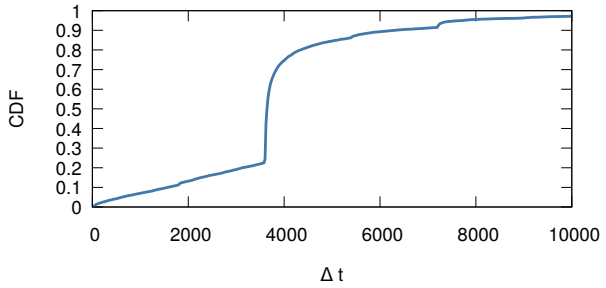



**Figure 3: ECDF of the median $\Delta t$ for recursives with at least 5 queries to ns1-ns5.dns.nl (TTL of 3600 s.)**

names.) We next examine this question for specific records in .nl, the country code domain (ccTLD) for the Netherlands and the Root (.) DNS zone. With traffic from "the wild" and a measurement target used by millions, this section uses a domain popular enough to stay in-cache at recursives.

## 4.1 Requests at .nl's Authoritatives

We apply this methodology to data for .nl country-code top-level domain (ccTLD). We look specifically at the A-records for the nameservers of .nl, ns[1-5].dns.nl.

**Methodology:** We use passive observations of traffic to the .nl authoritative servers.

For each target name in the zone and source (some recursive server, identified by IP address), we build a timeseries of all requests and compute their interarrival time, $\Delta$. Following the classification from §3.4, we label queries as: **AC** if $\Delta < TTL$, showing an unnecessary query to the authoritative; **AA** if $\Delta \geq TTL$, an expected or delayed cache refresh. (We do not see cache hits and so there are no **CC** events.)

**Dataset:** At the time of our analysis (February 2018) there were 8 authoritative servers for the .nl zone. We collect traffic for the 4 unicast and one anycast authoritative servers, and store the data in ENTRADA [51] for analysis.

Since our data for .nl is incomplete, and we know recursives will query all authoritatives over time [24], our analysis represents a conservative estimate of TTL violations—we expect to miss some CA-type queries from resolvers to non-monitored authoritatives.

We collect data for a period of six hours on 2018-02-22 starting at 12:00 UTC. We only evaluate recursives that sent at least five queries for our domains of interest, omitting infrequent recursives (they do not change results noticeably). We discard duplicate queries, for example, a few retransmissions (less than 0.01% of the total queries). In total, we consider more than 485k queries from 7,779 different recursives.

**Results:** Figure 3 shows the distribution of $\Delta t$ that we observe in our measurements, reporting the median $\Delta t$ for any resolver that sends at least 5 queries.

About 28% of queries are frequent, with an inter-arrival less than 10 s, and 32% of these are sent to multiple authoritatives. We believe these are due to recursives submitting queries in parallel to speed up replies (perhaps the "Happy Eyeballs" algorithm [42])

Since these closely-timed queries are not related to recursive caching, we exclude them from analysis. The remaining data is 348k queries from 7,703 different recursives.

The largest peak is at 3600 s, what was expected: the name was queried and cached for the full hour TTL, then the next request causes the name to be re-fetched. These queries are all of type AA.

The smaller peak around 1800 s, as well as queries with other times less than 3600 s, correspond to type AC-queries—queries that could have been supplied from the cache but were not. 22% of resolvers sent most of their queries within an time interval that is less than 3600 s or even more frequent. These AC queries occur because of TTL limiting, cache fragmentation, or other reasons that clear the cache.

## 4.2 Requests at the DNS Root

In this section we perform a similar analysis as for §4.1, in which we look into DNS queries received at all Root DNS servers (except G-Root), and create a distribution of the number of queries received per source IP address (*i.e.,* per recursive).

In this analysis we use data from the DITL (*Day In The Life*) dataset of 2017, available at DNS-OARC [8]. We look at all DNS queries received for the DS record of the domain nl, received at the Root DNS servers along the entire day on April 12, 2017 (UTC). This dataset consists of queries from more than 70.3k unique recursives seen across all Root servers. Note that the DS record for nl has a TTL of 86400 seconds (24 hours). That is, in theory, one could expect to see just one query per recursive arriving at a given root letter, for the DS record of nl within the 24-hour interval.

Each line in Figure 4 shows the distribution of the total number of queries received at the Root servers from individual recursives asking for the DS record of nl. Besides F- and H-Root, the distribution is similar across all Root servers; these are plotted in light-gray lines. F-Root shows the "most friendly" behavior from recursives, where around 5% of them sent 5 or more queries for nl. As opposed to F, H-Root (dotted red line) shows the "worst" behavior from recursives, where more than 10% of them sent 5 or more queries for nl within the 24-hour period.

The solid black line in Figure 4 shows the distribution for all the queries across all Root servers. The majority (around 87%) of recursives does send only one query within the 24-hour interval. However, considering all Root servers, we see around 13% of recursives that have sent multiple queries. Note that the distributions shown in Figure 4 have (very) long tails, and we see up to more than 21.8k queries from a single recursive within the 24-hour period for the nl DS record; *i.e.,* roughly one query every 4 seconds from the same IP address for the same DS record.

*Discussion:* we conclude that measurements of popular domains within .nl (§4.1) and the Roots (§4.2) show that about 63% and 87% of recursives honor the full TTL, respectively. These results are roughly in-line with our observations with RIPE Atlas (§3).

# 5 THE CLIENT'S VIEW OF AUTHORITATIVES UNDER DDOS

We next use controlled experiments to evaluate how DDoS attacks at authoritative DNS servers impacts client experience. Our studies of caching in controlled experiments (§3) and passive observations (§4) have shown that caching often works, but not always—about

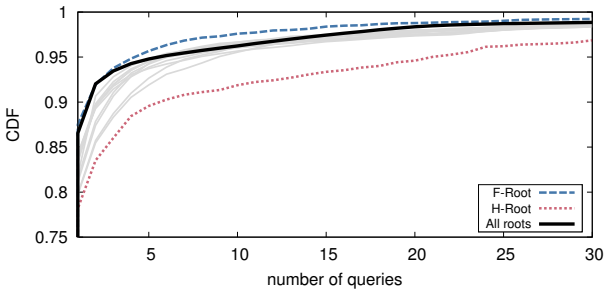



**Figure 4: Distribution of the number of queries for the DS record of nl received for each recursive. Dataset: DNS-OARC DITL on 2017-04-12t00:00Z for 24 hours. All Root servers with similar distributions are shown in light-gray lines.**

70% of controlled experiments and 30% of passive observations see full cache lifetimes. Since results of specific experiments vary, we sweep the space of attack intensities to understand the range of response from *complete* failure of authoritative servers, to *partial* failures.

## 5.1 Emulating DDoS

To emulate DDoS attacks we begin with the same test domain (cachetest.nl) we used for controlled experiments in §3. We run a normal DNS service for some time, querying from RIPE Atlas. After caches are warm, we then simulate a DDoS attack by dropping some fraction or all incoming DNS queries to each authoritative. (We drop incoming traffic randomly with Linux `iptables`. As such, packet drop is not biased towards any recursive.) After we begin dropping traffic, answers come either from caches at recursives or, for partial attacks, from a lucky query that passes through.

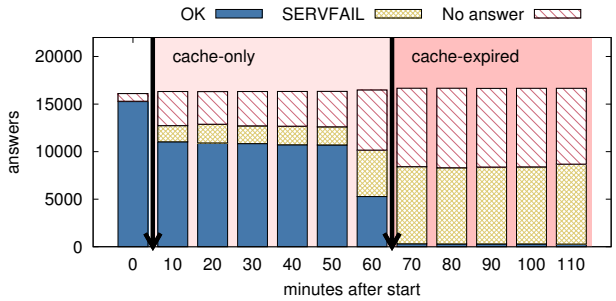
This emulation of DDoS captures traffic loss that occurs in DDoS attack as router queues overflow. This emulation is not perfect, since we simulate loss at the last hop-router, but in real DDoS attacks packets are often lost on access links near the target. Our emulation approximates this effect with one aggregate loss rate.

DDoS attacks are also accompanied by queueing delay, since buffers at and near the target are full. We do not model queueing delay, although we do observe latency increasing due to retries. In modern routers, queueing delay due to full router buffers should be less than the retry interval. In addition, observations during real-world DDoS events show that the few queries that are successful see response times that are not much higher than typical [21], suggesting that loss (and not delay) is the dominant effect of DDoS in practice. However, a study that adds queueing latency to the attack model is interesting future work.

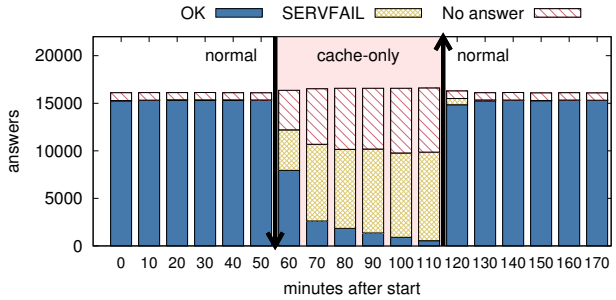
## 5.2 Clients During Complete Authoritatives Failure

We first evaluate the worst-case scenario for a DNS operator: complete unreachability of all authoritative name servers. Our goal is to understand when and for how long caches cover such an outage.

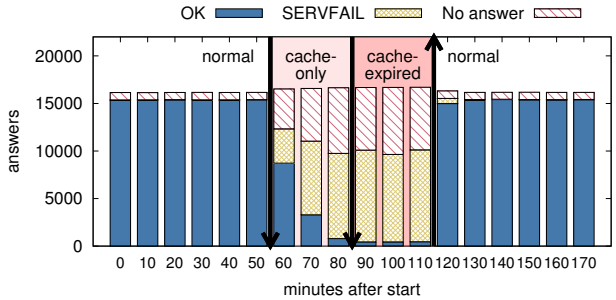
Table 4 shows Experiments A, B, and C which simulate complete failure. In Experiment A, each VP makes only one query before the DDoS begins. In Experiment B we allow several queries to take place, and Experiment C allows several queries with a shorter TTL.




**(a) Experiment A: 3600-10min-1down; arrows indicate DDoS start and cache expiration**




**(b) Experiment B: 3600-10min-1down-1up; arrows indicate DDoS start and recovery**




**(c) Experiment C: 1800-10min-1down-1up; arrows indicate DDoS start, cache expiration and recovery**

**Figure 5: Answers received during DDoS attacks.**

**Caches Protect Some:** We first consider Experiment A, with one query that warms the cache immediately followed by the attack. Figure 5a shows these responses over time, with the onset of the attack the first downward arrow between 0 and 10 minutes, and with the cache expired after the second downward arrow between 60 and 70 minutes. We see that after the DDoS starts but before the cache has fully expired (between the downward arrows) initially 30% and eventually 65% of queries fail with either no answer or a SERVFAIL error. While not good, this does mean that *35% to 70% of queries during the DDoS are successfully served from the cache.* By contrast, shortly after the cache expires, almost all queries fail (only 25 VPs or 0.2% of the total seem to provide stale answers).

**Caches Fill at Different Times:** In a more realistic scenario, VPs have filled their caches at different times. In Experiment A,

| | *Experiment Parameters* | | | | | | | *Results* | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | TTL in sec. | DDoS start | DDoS dur. | queries before | total dur. | probe interval | failure | Total probes | Valid probes | VPs | Queries | Total answers | Valid answers |
| A | 3600 | 10 | 60 | 1 | 120 | 10 | 100% (both NSes) | 9224 | 8727 | 15339 | 136423 | 76619 | 76181 |
| B | 3600 | 60 | 60 | 6 | 240 | 10 | 100% (both NSes) | 9237 | 8827 | 15528 | 357102 | 293881 | 292564 |
| C | 1800 | 60 | 60 | 6 | 180 | 10 | 100% (both NSes) | 9261 | 8847 | 15578 | 258695 | 199185 | 198197 |
| D | 1800 | 60 | 60 | 6 | 180 | 10 | 50% (one NS) | 9139 | 8708 | 15332 | 286231 | 273716 | 272231 |
| E | 1800 | 60 | 60 | 6 | 180 | 10 | 50% (both NSes) | 9153 | 8708 | 15320 | 285325 | 270179 | 268786 |
| F | 1800 | 60 | 60 | 6 | 180 | 10 | 75% (both NSes) | 9141 | 8727 | 15325 | 278741 | 259009 | 257740 |
| G | 300 | 60 | 60 | 6 | 180 | 10 | 75% (both NSes) | 9206 | 8771 | 15481 | 274755 | 249958 | 249042 |
| H | 1800 | 60 | 60 | 6 | 180 | 10 | 90% (both NSes) | 9226 | 8778 | 15486 | 269030 | 242725 | 241569 |
| I | 60 | 60 | 60 | 6 | 180 | 10 | 90% (both NSes) | 9224 | 8735 | 15388 | 253228 | 218831 | 217979 |

**Table 4: DDoS emulation experiments [35]; DDoS start, durations and probe interval are given in minutes.**

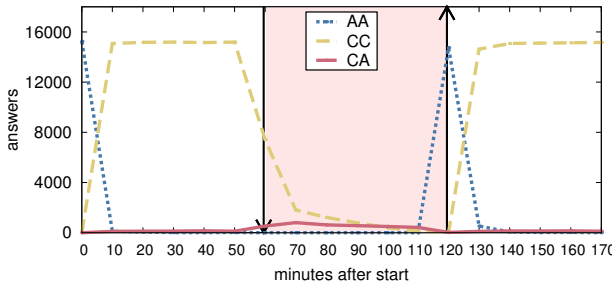



**Figure 6: Timeseries of answers for Experiment B**

caches are freshly filled and should last for a full hour after the start of attack. Experiment B is designed for the opposite and worst case: we begin warming the cache one hour before the attack and query 6 times from each VP. Other parameters are the same, with the attack lasting for 60 minutes (also the cache duration), but then we restore the authoritatives to service.

Figure 5b shows the results of Experiment B. While about 50% of VPs are served from the cache in the first 10 minute round after the DDoS starts, the fraction served drops quickly and is at only about 3% one hour later. Three factors are in play here: most caches were filled 60 minutes before the attack and are timing out in the first round. While the timeout and query rounds are both 60 minutes apart, Atlas intentionally spreads queries out over 5 minutes, so we expect that some queries happen after 59 minutes and others 61 minutes.

Second, we know some large recursives have fragmented caches (§3.5), so we expect that some of the successes between times 70 and 110 minutes are due to caches that were filled between times 10 and 50 minutes. This can actually be seen in Figure 6, where we show a timeseries of the answers for Experiment B, where we see CC (correct cache responses) between times 60 and 90.

Third, we see an increase in the number of CA queries that are answered by the cache with expired TTLs (Figure 6). This increase is due to servers serving stale content [17].

**Caches Eventually All Expire:** Finally, we carry out a third emulation but with half the cache lifetime (1800 s or 30 minutes rather than the full hour). Figure 5c shows response over time. These results are similar to Experiment B, with rapid fall-off when the attack starts as caches age. After the attack has been underway for 30 minutes all caches must have expired and we see only a few (about 2.6%) residual successes.

## 5.3 Discussion of Complete Failures

Overall we see that *caching is partially successful in protecting during a DDoS*. With full, valid caches, half or more VPs get service. However, caches are filled at different times and expire, so an operator cannot count on a full cache duration for any customers, even for popular ("always in the cache") domains. The protection provided by caches depends on their state in the recursive resolver, something outside the operator's control. In addition, our evaluation of caching in §3 showed that caches will end early for some VPs.

Second, we were surprised that a tiny fraction of VPs are successful after all caches should have timed out (after the 80 minutes period in Experiment A, and between 90 and 110 minutes in Experiment C). These successes suggest an early deployment of "serve stale", something currently under review in the IETF [17] is to serve a previously known record beyond its TTL if authoritatives are unreachable, with the goal of improving resilience under DDoS. We investigated the Experiment A, where see that 1048 answers of the 1140 successes in the second half of the outage. These successes are from 471 VPs (and 215 recursives), most of them answered by OpenDNS and Google public DNS servers, suggesting experimentation not yet widespread. Out of these 1048 queries, 1031 return a TTL value equals to 0, as specified in the IETF stale draft [17].

## 5.4 Client Reliability During Partial Authoritative Failure

The previous section examined DDoS attacks that result in complete failure of all authoritatives, but often DDoS attacks result in *partial failure*, with 50% or 90% packet loss at the authoritatives. (For example, consider the November 2015 DDoS attack on the DNS Root [21].) We next study experiments with partial failures, showing that *caching and retries together nearly fully protect 50% DDoS events, and protect half of VPs even during 90% events.*

We carry out several Experiments D to I in Table 4. We follow the procedure outlined in §5.1, looking at the DDoS-driven loss rates of 50%, 75%, and 90% with TTLs of 1800 s, 300 s and 60 s. Graphs omitted due to space can be found in an appendix of our technical report [23].

**Near-Full Protection from Caches During Moderate Attacks:** We first consider Experiment E, a "mild" DDoS with 50% loss, with VP success over time in Figure 7a. In spite of a loss rate

that would be crippling to TCP, nearly all VPs are successful in DNS. This success is due to two factors: first, we know that many clients are served from caches, as was shown in Experiment A with full loss (Figure 5a). Second, most recursives retry queries, so they recover from loss of a single packet and are able to provide an answer. Together, these mean that failures during the first 30 minutes of the event is 8.5%, slightly higher than the 4.8% fraction of failures before the DDoS. For this experiment, the TTL is 1800 s (30 minutes), so we might expect failures to increase halfway through the DDoS. We do not see any increase in failures because caching and retries are *synergistic*, a successful retried query will place the answer in a cache for a later query. The importance of this result is that *DNS can survive moderate-size attacks when caching is possible*. While a positive, retries do increase latency, something we study in §5.5.

**Attack Intensity Matters:** While clients do quite well with 50% loss at all authoritatives, failures increase with the intensity of the attack.

Experiments F and H, shown in Figure 7b and Figure 7c increase the loss rate to 75% and 90%. We see the number of failures increases to about 19.0% with 75% loss and 40.3% with 90% loss. It is important to note that *roughly 60% the clients are still served even with 90% loss.*

We also see that this level of success is consistent over the entire hour-long DDoS event, even though the cache duration is only 30 minutes. This consistency confirms the importance of caching and retries in combination.

To verify the effects of this interaction, Experiment I changes the caching duration to 60 s, less than one round or probing. Comparing Experiment I in Figure 7d to H in Figure 7c, we see that the failure rate increases from 30% to about 63%. However, even with no caching, about 37% of queries still are answered, due to resolvers that serve stale content and recursives retries. We investigate retries in §6.

## 5.5 Client Latency During Partial Authoritative Failure

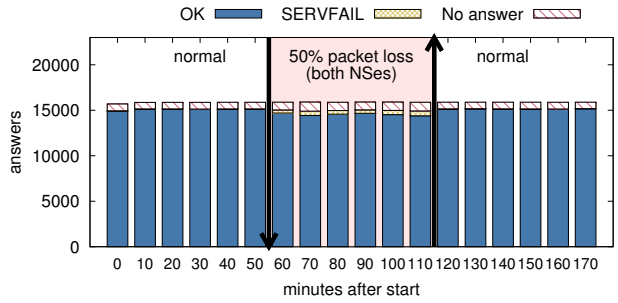
We showed that client reliability is higher than expected during failures (§5.4) due to a combination of caching and retries. We next consider client *latency*. Latency will increase during the DDoS because of retries and queueing delay, but we will show that latency increases less than one might expect due to caching.

To examine latency we return to Experiments D through I (Table 4), but look at latency (time to complete a query) rather than success. For these experiments clients timeout after 5 s.

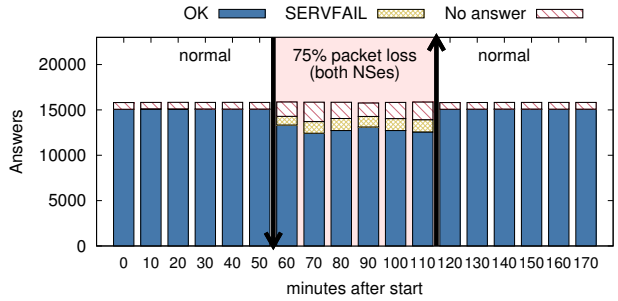
Figures 8a to 8d show latency during each emulated DDoS scenario (experiments with figures omitted here are in our technical report [23]). Latencies are not evenly distributed, since some requests get through immediately while others must be retried one or more times, so in addition to mean, we show 50, 75 and 90% quantiles to characterize the tail of the distribution.

We emulate DDoS by dropping requests (§5.1) and, hence, latencies reflect retries and loss, but not queueing delay, underrepresenting latency in real-world attacks. However, their shape (some low latency and a few long) is consistent with and helps explain what has been seen in the past [21].

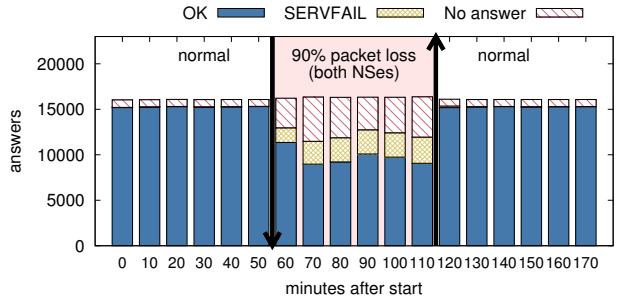
Beginning with Experiment E, the moderate attack in Figure 8a, we see *no* change to median latency. This result is consistent with



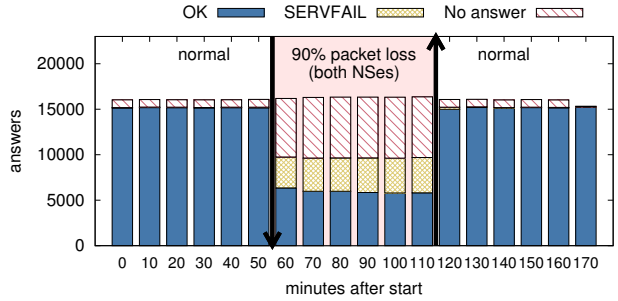

(a) Experiment E (1800-50p-10min): 50% packet loss




(b) Experiment F (1800-75p-10min): 75% packet loss

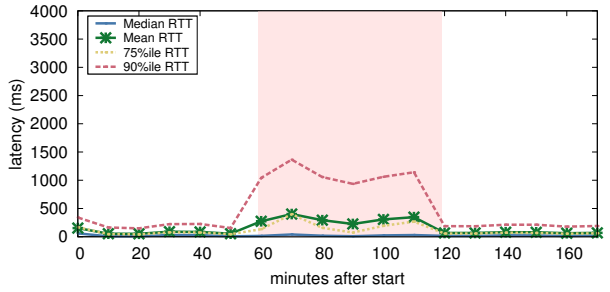



(c) Experiment H (1800-90p-10min): 90% packet loss

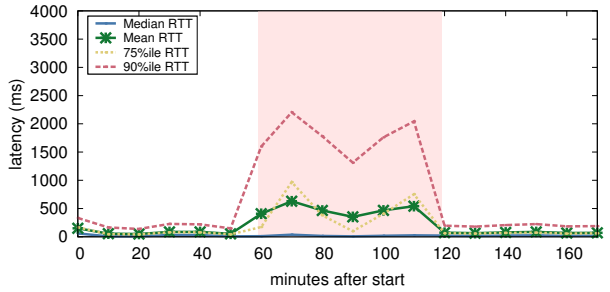



(d) Experiment I (60-90p-10min): 90% packet loss

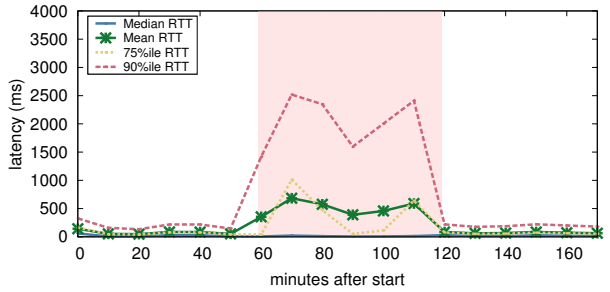
**Figure 7: Answers received during DDoS attacks; first and second vertical lines show start and end of DDoS.**



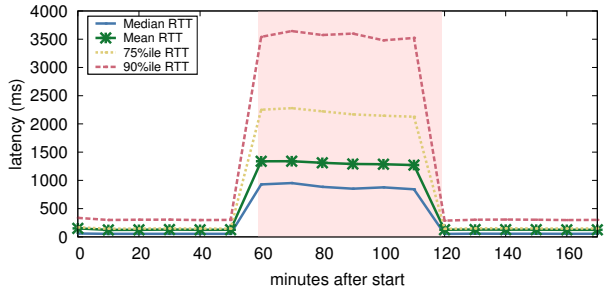

**(a) Experiment E: 50% packet loss (1800 s TTL)**




**(b) Experiment F: 75% packet loss (1800 s TTL)**




**(c) Experiment H: 90% packet loss(1800 s TTL)**




**(d) Experiment I: 90% packet loss (60 s TTL)**

**Figure 8: Latency results; Shaded area indicates the interval of an ongoing DDoS attack.**

many queries being handled by the cache, and half of those not handled by the cache getting through anyway. We do see higher latency in the 90%ile tail, reflecting successful retries. This tail also increases the mean some.

This trend increases in Experiment F in Figure 8b, where 75% of queries are lost. Now we see the 75%ile tail has increased, as has the number of unanswered queries, and the 90%ile is twice as long as in Experiment E.

We see the same latency in Experiment H with DDoS causing 90% loss. We set the timeouts to 5 s, so the larger attack results in more unsuccessful queries, but latency for successful queries is not much worse than with 75% loss. Median latency is still low due to cached replies.

Finally, Experiment I greatly reduces opportunities for caching by reducing cache lifetime to one minute. Figure 8d shows that loss of caching increases median RTT and significantly increases the tail latency. Compared with Figure 8c (same packet loss ratio but 1800 s TTL), we can clearly see the benefits of caching in terms of latency (in addition to reliability): a half-hour TTL value reduced the latency from 1300 ms to 390 ms. Longer TTLs also help reduce tail latency relative to shorter TTLs (compare, for example, the 90%ile RTT in Experiments I vs. H in Figure 8).

**Summary:** DDoS effects often increase client latency. For moderate attacks, increased latency is seen only by a few "unlucky" clients whose do not see a full cache and whose queries are lost. Caching has an important role in reducing latency during DDoS, but while it can often mitigate most reliability problems, it cannot avoid latency penalties for all VPs. Even when caching is not available, roughly 40% of clients get an answer, either by serving stale or retries as we investigate next.

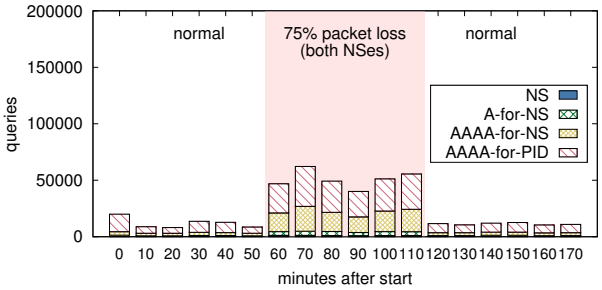
## 6 THE AUTHORITATIVE'S PERSPECTIVE

Results of partial DDoS events (§5.4) show that DNS is surprisingly reliable—even with a DDoS resulting in 90% packet loss and lasting longer than the cache timeout, more than half of VPs get answers with 30 minute caches (Figure 7c), and about 40% of VPs get answers (Figure 7d) even with minimal duration caches. These results are due to a combination of caching and retries. We next examine this from the perspective of the authoritative server.

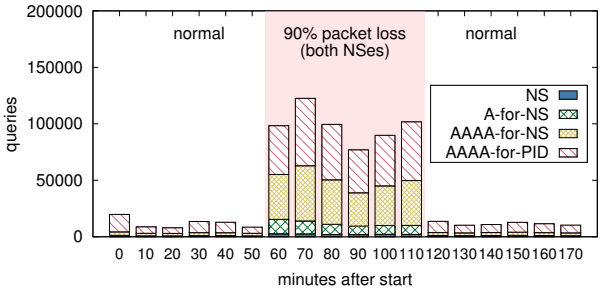
### 6.1 Recursive-Authoritative Traffic during a DDoS

We first ask: are retries by recursive resolvers responsible for the success rates observed in §5.4? To investigate this question, we return the partial DDoS experiments and look at how many queries are sent to the authoritative servers. We measure queries before they are dropped by our simulated DDoS. Recursives must make multiple queries to resolve a name. We break out each type of query: for the nameserver (NS), the nameserver's IPv4 and v6 addresses (A-for-NS and AAAA-for-NS), and finally the desired query (AAAA-for-PID). Note that the authoritative is IPv4 only, so AAAA-for-NS is non-existent and subject to negative caching, while the other records exist and use regular caching.

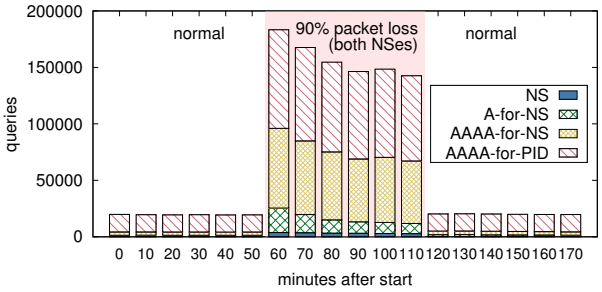
We begin with the DDoS causing 75% loss in Figure 9a. For this experiment, we observe 18,407 unique IP addresses of recursives ($R_n$) querying for AAAA records directly to our authoritatives. During the DDoS, queries increase by about 3.5×. We expect 4




**(a) Experiment F: 1800-75p-10min, 75% packet loss**




**(b) Experiment H: 1800-90p-10min, 90% packet loss**




**(c) Experiment I: 60-90p-10min, 90% packet loss**

**Figure 9: Number of received queries by the authoritative servers. Shaded area indicates the interval of an ongoing DDoS attack.**

trials, since the expected number of tries until success with loss rate $p$ is $(1-p)^{-1}$. For this scenario, results are cached for up to 30 minutes, so successful queries are reused in recursive caches. This increase occurs both for the target AAAA record, and also for the non-existent AAAA-for-NS records. Negative caching for our zone is configured to 60 s, making caching of NXDOMAINs for AAAA-for-NS less effective than positive caches.

The offered load on the server increases further with more loss (90%), as shown in Experiment H (Figure 9b). The higher loss rate results in a much higher offered load on the server, average 8.2× normal.

Finally, in Figure 9c we reduce the effects of caching at a 90% DDoS and with a TTL of 60 s. Here we see also about 8.1× more queries at the server before the attack. Comparing this case to Experiment H, caching reduces the offered load on the server by about 40%.

**Implications:** The implication of this analysis is that legitimate clients "hammer" with retries the already-stressed server during a DDoS. For clients, retries are important to get reliability; and each client independently chooses to retry.

The server is already under stress due to the DDoS, so these retries add to that stress. However, the DDoS traffic is almost certainly much larger than the retried of legitimate traffic. (A server experiencing a volumetric attack causing 90% loss must be receiving 10× its capacity. Regular traffic is a small fraction of normal capacity, so even 4× regular is still much less than the attack traffic.) The multiplier for retried legitimate traffic depends on the implementations stub and recursive resolver, as well as application-level retries and defection (users hitting reload in their browser, and later giving up). Our experiment omits application-level retries and likely gives a lower bound. We next examine specific recursive implementations to see their behavior.

## 6.2 Sources of Retries: Software and Multi-level Recursives

Experiments in the prior section showed that recursive resolvers "hammer" authoritatives when queries are dropped. We reexamine DNS software (since 2012 [52]), and additionally show deployments amplify retries.

**Recursive Software:** Prior work showed that recursive servers retry many times when an authoritative is unresponsive [52], with evaluation of BIND 9.7 and 9.8, DNSCache, Unbound, WindowsDNS and PowerDNS. We studied retries in BIND 9.10.3 and Unbound 1.5.8 to quantify the number of retries. Examining only requests for AAAA records, we see that normal requests with a responsive authoritative ask for the AAAA records for all authoritatives and the target name (3 total requests when there are 2 authoritatives). When all authoritatives are unavailable, we see about 7× more requests before the recursives time out. (Exact numbers vary in different runs, but typically each request is made 6 or 7 times.) Such retries are appropriate, provided they are paced (both use exponential backoff), they explain part of the increase in legitimate traffic during DDoS events. Full data is in an appendix of our technical report [23].

**Recursive Deployment:** Another source of extra retries is complex recursive deployments. We showed that operators of large recursives often use complex, multi-level resolution infrastructure (§3.5). This infrastructure can amplify the number of retries during reachability problems at authoritatives.

To quantify amplification, we count both the number of *Rn* recursives and AAAA queries for each probe ID reaching our authoritatives. Figure 10 show the results for Experiment I. These values represent the amplification in two ways: during stress, more *Rn* recursives will be used for each probe ID and these *Rn* will generate more queries to the already stressed authoritatives. As the figures show, the median number of *Rn* recursives employed doubles (from 1 to 2) during the DDoS event, as does the 90%ile (from 2 to 4). The maximum rises to 39. The number of queries for each probe ID grows more than 3×, from 2 to 7. Worse, the 90%ile grows more than 6× (3 queries to 18). The maximum grows 53.5×, reaching up to 286 queries for one single probe ID. This value, however, is a lower bound, given there are a large number of A and AAAA

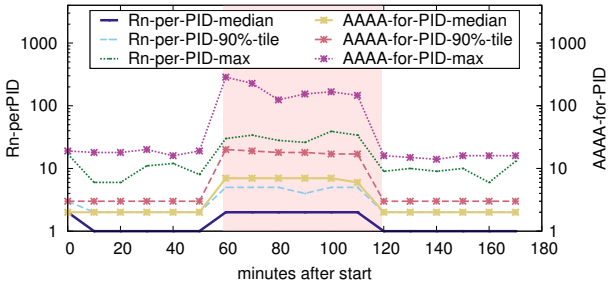



**Figure 10: *Rn* recursives and AAAA queries used in Experiment I, normalized by the number of probe IDs.**

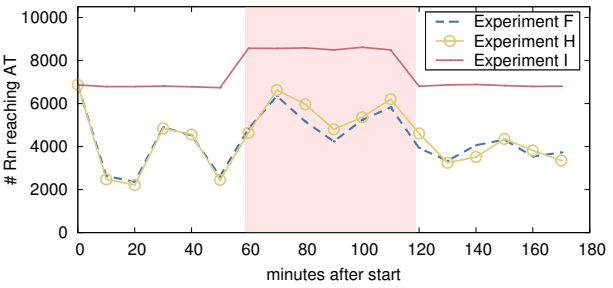



**Figure 11: Unique *Rn* recursives addresses observed at authoritatives**

queries that ask for NS records and not the probe ID (AAAA and A-for NS in Figure 9).

We can also look at the aggregate effects of retries created by the complex recursive infrastructure. Figure 11 shows the timeseries of unique IP addresses of $R_n$ observed at the authoritatives. Before the DDoS period, for Experiment I with TTL of 60 s, we see a constant number of recursives reaching our authoritatives; *i.e.,* all queries should be answered by authoritatives (no caching at this TTL value). For experiments F and H, both with TTL of 1800 s, the number of recursives reaching our authoritative oscillates before the DDoS; peaks are observed when caches expire as expected.

During the DDoS we observe a similar behavior for all three experiments in Figure 11: as packets are dropped at the authoritative (at rates of 75, 90 and 90% for F, H, and I respectively) we see an increase on the number of *Rn* recursives querying our authoritatives; for experiments F and H we see drops when caching is expected, but not for experiment I. The reason for this behavior is that the underlying layer of recursives starts forwarding queries to other recursives, which is amplified in the end. (We show this behavior for an individual probe in our technical report [23], where we observe the growth in the number of queries received at the authoritatives and the number of recursives used.)

Most complex resolution infrastructures are proprietary (as far as we know only one study has examined them [45]), so we cannot make recommendations about how large recursive resolvers ought to behave. We suggest that the aggregate traffic of large recursive resolvers should strive to be within a constant factor of single recursives, perhaps a factor of 4. We also encourage additional study of large recursive resolvers, and their operators to share information about their behavior.

## 7 RELATED WORK

**Caching by Recursives:** Several groups have shown that DNS caching can be imperfect. Hao and Wang analyzed the impact of nonce domains on DNS recursive's caches [11]. Using two weeks of data from two universities they showed that filtering one-time domains improves cache hit rates. In two studies, Pang *et al.* [28, 29] reported that web clients and local recursives do not always honor TTL values provided by authoritatives. Almeida *et al.* [2] analyzed DNS traces of a mobile operator, and used a mobile application to see TTLS in practice. They find that most domains have short TTLs (less than 60 s), and report and evidence of TTL manipulation by recursives. Schomp *et al.* [45] demonstrate widespread use of multi-level recursives by large operators, as well as TTL manipulation. Our work builds on this prior work, examining caching and TTL manipulation systematically and considering its effects on resilience.

**DNS client behavior:** Yu *et al.* investigated how stubs and recursives select authoritative servers, and were the first to demonstrate the large number of retries when all authoritatives are unavailable [52]. We also investigated how recursives select authoritative servers in the wild and found that recursives tend to prefer authoritatives with shorter latency, but query all authoritatives for diversity [24]. We confirm Yu's work and focus on authoritative selection during DDoS from several perspectives.

**Authoritatives during DDoS:** We investigated how the Root DNS service behaved during the Nov. 2015 DDoS attacks [21]. This report focuses on the interactions of IP anycast and both latency and reachability, as seen from RIPE Atlas. Rather than look at aggregate behavior and anycast, our methodology here examines how clients interact with their recursive resolvers, while this prior work focused on authoritatives only, bypassing recursives. In addition, here we have full access to clients and authoritatives traffic during our experiments, and we evaluate DDoS with controlled loss rates. The prior study has incomplete data and focuses on specific results of two events. These differences stem from their study of natural experiments from real-world events and our controlled experiments.

## 8 IMPLICATIONS

We evaluated DNS resilience, showing that caches and retries can mitigate much of the harm from a DDoS attack, provided the cache is full and some requests can get to authoritative servers. The key implication of our study is to explain differences in the outcome of recent DDoS attacks.

Recent DDoS attacks on DNS services have seen very different outcomes for users. The Root Server System was a target in Nov. 2015 [38] and June 2016 [39]. The DNS Root has 13 *letters*, each an authoritative "server" implemented with some or many IP anycast instances. Analysis of these DDoS events showed that their effects were uneven across letters: for some, most or all anycast instances showed high loss, while other letters showed little or no loss [21]. However, the Root Operators state "There are no known reports of end-user visible error conditions during, and as a result

of, this incident. Because the DNS protocol is designed to cope with partial reachability…" [38].

In Oct. 2016, a much larger attack was directed at Dyn, a provider of DNS service for many second-level domains [12]. Although Dyn has a capable infrastructure and immediately took steps to address service problems, there were reports of user-visible service disruption in the technical and even popular press [31]. Reports describe intermittent failure of prominent websites including "Twitter, Netflix, Spotify, Airbnb, Reddit, Etsy, SoundCloud and The New York Times", each a direct or indirect customer of Dyn at the time.

Our work helps explain these very different outcomes. The Root DNS saw few or no user-visible problems because data in the root zone is cachable for a day or more, and because multiple letters and many anycast instances were continuously available. (All measurements in this paragraph are as of 2018-05-22.) Records in the root zone have TTLs of 1 to 6 days, and www.root-servers.org reports 922 anycast instances operating across the 13 authoritative servers. Dyn also operates a large infrastructure (https://dyn.com/dns/network-map/ reports 20 "facilities"), and faced a larger attack (reports of 1.2 Tb/s [44], compared to estimates of 35 Gb/s for the Nov. 2015 root attack [21]). But a key difference is *all* of the Dyn's customers listed above use DNS-based CDNs (for a description, see [7]) with multiple, Dyn-hosted DNS components with TTLs that range from 120 to 300 s.

In addition to explaining the effects, our experiments help get to the root causes behind these outcomes. Users of the Root benefited from caching and saw performance like Experiment E (Figure 7a), because root contents (TLDs like .com and country codes) are popular and certainly cached in recursives, and because some root letters were always available to refresh caches (either through a successful normal query, or a retry). By contrast, users requiring domains with very short TTLs (like the websites that had problems) receive performance more like Experiment I (Figure 7d) or Experiment C (Figure 5c). Even when some requests succeed an cache a popular name, short TTLs cause caches to clear quickly.

This example shows the importance of DNS's multiple methods of resilience (caching, retries, and at least some availability at one authoritative). It suggests that CDN operators may wish to consider longer timeouts to allow caching to help and give DNS operators deploy defenses. Experiment H suggests 30 minutes, Figure 7c.

Configuring short TTLs serves a role in CDNs that use DNS to direct clients to different application-level servers. Short TTLs allow for re-provisioning during DDoS attacks on web servers, but that leaves DNS servers vulnerable. This tension suggests traffic scrubbing by routing changes with long DNS TTLs may be preferred to short DNS TTLs, so that both layers can be robust. However, the complexity of interactions between DNS at multiple levels and CDNs suggests that more study is needed before recommending specific settings.

Finally, this evaluation helps complete our picture of DNS latency and reliability for DNS services that may consist of multiple authoritatives, some or all using IP anycast with multiple sites. To minimize latency, prior work has shown a single authoritative using IP anycast should maximize geographic dispersion of sites [43]. The latency of an overall DNS service with *multiple* authoritatives can be limited by the one with largest latency [24]. Prior work about resilience to DDoS attack has shown that individual IP anycast sites will suffer under DDoS as a function of the attack traffic that site receives relative to its capacity [21]. We show that the overall reliance of a DNS service composed of multiple authoritatives using IP anycast tends to be as resilient as the *strongest* individual authoritative. The reason for these opposite results is that, in both cases, recursive resolvers will try *all* authoritatives of a given service. For latency, they will sometimes choose a distant authoritative, but for resilience, they will continue until they find the most available authoritative.

## 9 CONCLUSIONS

This paper represents the first study of how the DNS resolution system behaves when authoritative servers are under DDoS attack. Caching and retries at recursive resolvers are key factors in this behavior. We show that together, caching and retries by recursive resolvers greatly improve the resilience of the DNS as a whole. In fact, they can largely cover over partial DDoS attacks for many users—even with a DDoS resulting in 90% packet loss and lasting longer than the cache timeout, more than half of VPs get answers with 30 minute caches (Figure 7c), and about 40% of VPs get answers (Figure 7d) even with minimal duration caches.

The primary cost of DDoS for users can be greater latency, but even this penalty is uneven across users, with a few getting much greater latency while some see no or little change. Finally, we show that one result retries is that traffic from legitimate users to authoritatives greatly increases (up to 8×) during service interruption, and that this effect is magnified by complex, multi-layer recursive resolver systems. The key outcome of work is to quantify the importance of caching and retries in recursives to resilience, encouraging use of at least moderate TTLs wherever possible.

## Acknowledgments

The authors would like to thank Jelte Jansen, Benno Overeinder, Marc Groeneweg, Wes Hardaker, Duanne Wessels, Warren Kumari, Stéphane Bortzmeyer, Maarten Aertsen, Paul Hoffman, our shepherd Mark Allman, and the anonymous IMC reviewers for their valuable comments on paper drafts.

This research has been partially supported by measurements obtained from RIPE Atlas, an open measurements platform operated by RIPE NCC, as well as by the DITL measurement data made available by DNS-OARC.

Giovane C. M. Moura, Moritz Müller, and Marco Davids developed this work as part of the SAND project (http://www.sand-project.nl).

John Heidemann's research is partially sponsored by the Air Force Research Laboratory and the Department of Homeland Security under agreements number FA8750-17-2-0280 and FA8750-17-2-0096. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes notwithstanding any copyright notation thereon.

## REFERENCES

[1] 1.1.1.1. 2018. The Internet's Fastest, Privacy-First DNS Resolver. https://1.1.1.1/. https://1.1.1.1/

[2] Mario Almeida, Alessandro Finamore, Diego Perino, Narseo Vallina-Rodriguez, and Matteo Varvello. 2017. Dissecting DNS Stakeholders in Mobile Networks. In *Proceedings of the 13th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT '17)*. ACM, New York, NY, USA, 28–34. https://doi.org/10.1145/3143361.3143375

[3] Manos Antonakakis, Tim April, Michael Bailey, Matt Bernhard, Elie Bursztein, Jaime Cochran, Zakir Durumeric, J. Alex Halderman, Luca Invernizzi, Michalis Kallitsis, Deepak Kumar, Chaz Lever, Zane Ma, Joshua Mason, Damian Menscher, Chad Seaman, Nick Sullivan, Kurt Thomas, and Yi Zhou. 2017. Understanding the Mirai Botnet. In *Proceedings of the 26th USENIX Security Symposium*. USENIX, Vancouver, BC, Canada, 1093–1110. https://www.usenix.org/system/files/conference/usenixsecurity17/sec17-antonakakis.pdf

[4] Arbor Networks. 2012. *Worldwide Infrastructure Security Report.* Technical Report 2012 Volume VIII. Arbor Networks. http://www.arbornetworks.com/resources/infrastructure-security-report

[5] Vaibhav Bajpai, , Steffie Eravuchira, Jürgen Schönwälder, Robert Kisteleki, and Emile Aben. 2017. Vantage Point Selection for IPv6 Measurements: Benefits and Limitations of RIPE Atlas Tags. In *IFIP/IEEE International Symposium on Integrated Network Management (IM 2017).* Lisbon, Portugal.

[6] Vaibhav Bajpai, Steffie Jacob Eravuchira, and Jürgen Schönwälder. 2015. Lessons Learned from using the RIPE Atlas Platform for Measurement Research. *SIGCOMM Comput. Commun. Rev.* 45, 3 (July 2015), 35–42. http://www.sigcomm.org/sites/default/files/ccr/papers/2015/July/0000000-0000005.pdf

[7] Matt Calder, Ashley Flavel, Ethan Katz-Bassett, Ratul Mahajan, and Jitendra Padhye. 2015. Analyzing the Performance of an Anycast CDN. In *Proceedings of the ACM Internet Measurement Conference.* ACM, Tokyo, Japan. https://doi.org/10.1145/2815675.2815717

[8] DNS OARC. 2018. DITL Traces and Analysis. https://www.dns-oarc.net/index.php/oarc/data/ditl/2018.

[9] R. Elz, R. Bush, S. Bradner, and M. Patton. 1997. Selection and Operation of Secondary DNS Servers. RFC 2182 (Best Current Practice). , 11 pages. https://doi.org/10.17487/RFC2182

[10] Google. 2018. Public DNS. https://developers.google.com/speed/public-dns/. https://developers.google.com/speed/public-dns/

[11] Shuai Hao and Haining Wang. 2017. Exploring Domain Name Based Features on the Effectiveness of DNS Caching. *SIGCOMM Comput. Commun. Rev.* 47, 1 (Jan. 2017), 36–42. https://doi.org/10.1145/3041027.3041032

[12] Scott Hilton. 2016. Dyn Analysis Summary Of Friday October 21 Attack. Dyn blog https://dyn.com/blog/dyn-analysis-summary-of-friday-october-21-attack/.

[13] Paul Hoffman, Andrew Sullivan, and K. Fujiwara. 2018. DNS Terminology. Internet Draft. https://datatracker.ietf.org/doc/draft-ietf-dnsop-terminology-bis/?include_text=1

[14] ICANN. 2014. RSSAC002: RSSAC Advisory on Measurements of the Root Server System. https://www.icann.org/en/system/files/files/rssac-002-measurements-root-20nov14-en.pdf.

[15] ISC BIND. 2018. Chapter 6. BIND 9 Configuration Reference. https://ftp.isc.org/isc/bind9/cur/9.10/doc/arm/Bv9ARM.ch06.html.

[16] Sam Kottler. 2018. February 28th DDoS Incident Report | Github Engineering. . https://githubengineering.com/ddos-incident-report/.

[17] D. Lawrence and W. Kumari. 2017. Serving Stale Data to Improve DNS Resiliency-02. Internet Draft. https://www.ietf.org/archive/id/draft-tale-dnsop-serve-stale-02.txt

[18] P.V. Mockapetris. 1987. Domain names - concepts and facilities. RFC 1034 (Internet Standard). , 55 pages. https://doi.org/10.17487/RFC1034 Updated by RFCs 1101, 1183, 1348, 1876, 1982, 2065, 2181, 2308, 2535, 4033, 4034, 4035, 4343, 4035, 4592, 5936, 8020.

[19] P.V. Mockapetris. 1987. Domain names - implementation and specification. RFC 1035 (Internet Standard). , 55 pages. https://doi.org/10.17487/RFC1035 Updated by RFCs 1101, 1183, 1348, 1876, 1982, 1995, 1996, 2065, 2136, 2181, 2137, 2308, 2535, 2673, 2845, 3425, 3658, 4033, 4034, 4035, 4343, 5936, 5966, 6604, 7766.

[20] Carlos Morales. 2018. February 28th DDoS Incident Report | Github EngineeringNETSCOUT Arbor Confirms 1.7 Tbps DDoS Attack; The Terabit Attack Era Is Upon Us. https://www.arbornetworks.com/blog/asert/netscout-arbor-confirms-1-7-tbps-ddos-attack-terabit-attack-era-upon-us/.

[21] Giovane C. M. Moura, Ricardo de O. Schmidt, John Heidemann, Wouter B. de Vries, Moritz Müller, Lan Wei, and Christian Hesselman. 2016. Anycast vs. DDoS: Evaluating the November 2015 Root DNS Event. In *Proceedings of the ACM Internet Measurement Conference.* https://doi.org/10.1145/2987443.2987446

[22] Giovane C. M. Moura, John Heidemann, Moritz Müller, Ricardo de O. Schmidt, and Marco Davids. 2018. Datasets from "When the Dike Breaks: Dissecting DNS Defenses During DDoS". (May 2018). Web page https://ant.isi.edu/datasets/dns/Moura18a_data.

[23] Giovane C. M. Moura, John Heidemann, Moritz Müller, Ricardo de O. Schmidt, and Marco Davids. 2018. *When the Dike Breaks: Dissecting DNS Defenses During DDoS (extended).* Technical Report ISI-TR-725b. USC/Information Sciences Institute. https://www.isi.edu/%7ejohnh/PAPERS/Moura18a.html (updated Sept. 2018).

[24] Moritz Müller, Giovane C. M. Moura, Ricardo de O. Schmidt, and John Heidemann. 2017. Recursives in the Wild: Engineering Authoritative DNS Servers. In *Proceedings of the ACM Internet Measurement Conference.* London, UK, 489–495. https://doi.org/10.1145/3131365.3131366

[25] NL Netlabs. 2018. NL Netlabs Documentation - Unbound - undbound.conf.5. https://nlnetlabs.nl/documentation/unbound/unbound.conf/.

[26] OpenDNS. 2018. Setup Guide: OpenDNS. https://www.opendns.com/setupguide/. https://www.opendns.com/setupguide

[27] Jianping Pan, Y Thomas Hou, and Bo Li. 2003. An overview of DNS-based server selections in content distribution networks. *Computer Networks* 43, 6 (2003), 695–711.

[28] Jeffrey Pang, Aditya Akella, Anees Shaikh, Balachander Krishnamurthy, and Srinivasan Seshan. 2004. On the Responsiveness of DNS-based Network Control. In *Proceedings of the 4th ACM SIGCOMM Conference on Internet Measurement (IMC '04).* ACM, New York, NY, USA, 21–26. https://doi.org/10.1145/1028788.1028792

[29] Jeffrey Pang, James Hendricks, Aditya Akella, Roberto De Prisco, Bruce Maggs, and Srinivasan Seshan. 2004. Availability, Usage, and Deployment Characteristics of the Domain Name System. In *Proceedings of the 4th ACM SIGCOMM Conference on Internet Measurement (IMC '04).* ACM, New York, NY, USA, 1–14. https://doi.org/10.1145/1028788.1028790

[30] Paul Vixie and Gerry Sneeringer and Mark Schleifer. 2002. Events of 21-Oct-2002. http://c.root-servers.org/october21.txt.

[31] Nicole Perlroth. 2016. Hackers Used New Weapons to Disrupt Major Websites Across U.S. *New York Times* (Oct. 22 2016), A1. http://www.nytimes.com/2016/10/22/business/internet-problems-attack.html

[32] Nicole Perlroth. 2016. Tally of Cyber Extortion Attacks on Tech Companies Grows. New York Times Bits Blog, http://bits.blogs.nytimes.com/2014/06/19/tally-of-cyber-extortion-attacks-on-tech-companies-grows/.

[33] Alec Peterson. 2017. EC2 resolver changing TTL on DNS answers? Post on the DNS-OARC dns-operations mailing list, https://lists.dns-oarc.net/pipermail/dns-operations/2017-November/017043.html.

[34] Quad9. 2018. Quad9 | Internet Security & Privacy In a Few Easy Steps. https://quad9.net.

[35] RIPE NCC. 2017. RIPE Atlas Measurement IDS. https://atlas.ripe.net/measurements/ID. ID is the experiment ID: TTL60: 10443671, TTL1800: 10507676, TTL3600: 10536725, TTL86400: 10579327, TTL3600-10min: 10581463, A:10859822, B: 11102436, C :11221270, D:11804500, E: 11831403, F: 11831403, G: 12131707, H:12177478 , I: 12209843.

[36] RIPE NCC Staff. 2015. RIPE Atlas: A Global Internet Measurement Network. *Internet Protocol Journal (IPJ)* 18, 3 (Sep 2015), 2–26.

[37] RIPE Network Coordination Centre. 2018. RIPE Atlas - Raw data structure documentations,https://atlas.ripe.net/docs/data_struct/.

[38] Root Server Operators. 2015. Events of 2015-11-30. http://root-servers.org/news/events-of-20151130.txt.

[39] Root Server Operators. 2016. *Events of 2016-06-25.* Technical Report. Root Server Operators. http://www.root-servers.org/news/events-of-20160625.txt

[40] Root Server Operators. 2017. Root DNS. http://root-servers.org/.

[41] José Jair Santanna, Roland van Rijswijk-Deij, Rick Hofstede, Anna Sperotto, Mark Wierbosch, Lisandro Zambenedetti Granville, and Aiko Pras. 2015. Booters—An Analysis of DDoS-as-a-Service Attacks. In *Proceedings of the 14th IFIP/IEEE Interatinoal Symposium on Integrated Network Management.* IFIP, Ottowa, Canada.

[42] D. Schinazi and T. Pauly. 2017. *Happy Eyeballs Version 2:Better Connectivity Using Concurrency.* RFC 8305. Internet Request For Comments. https://doi.org/10.17487/RFC8305

[43] Ricardo de O. Schmidt, John Heidemann, and Jan Harm Kuipers. 2017. Anycast Latency: How Many Sites Are Enough?. In *Proceedings of the Passive and Active Measurement Workshop.* Springer, Sydney, Australia, 188–200. http://www.isi.edu/%7ejohnh/PAPERS/Schmidt17a.html

[44] Bruce Schneier. 2016. Lessons From the Dyn DDoS Attack. blog https://www.schneier.com/essays/archives/2016/11/lessons_from_the_dyn.html.

[45] Kyle Schomp, Tom Callahan, Michael Rabinovich, and Mark Allman. 2013. On measuring the client-side DNS infrastructure. In *Proceedings of the 2015 ACM Conference on Internet Measurement Conference.* ACM, 77–90.

[46] Somini Sengupta. 2012. After Threats, No Signs of Attack by Hackers. *New York Times* (Apr. 1 2012), A1. http://www.nytimes.com/2012/04/01/technology/no-signs-of-attack-on-internet.html

[47] SIDN Labs. 2017. .nl stats and data. http://stats.sidnlabs.nl.

[48] Matthew Thomas and Duane Wessels. 2015. A study of caching behavior with respect to root server TTLs. DNS-OARC. https://indico.dns-oarc.net/event/24/contributions/374/

[49] Unbound. 2018. Unbound Documentation. https://www.unbound.net/documentation/unbound.conf.html.

[50] Weinberg, M., Wessels, D. 2016. Review and analysis of attack traffic against A-root and J-root on November 30 and December 1, 2015. In: DNS OARC 24 – Buenos Aires, Argentina. https://indico.dns-oarc.net/event/22/session/4/contribution/7.

[51] Maarten Wullink, Giovane CM Moura, Moritz Müller, and Cristian Hesselman. 2016. ENTRADA: A high-performance network traffic data streaming warehouse. In *Network Operations and Management Symposium (NOMS), 2016 IEEE/IFIP.* IEEE, 913–918.

[52] Yingdi Yu, Duane Wessels, Matt Larson, and Lixia Zhang. 2012. Authority Server Selection in DNS Caching Resolvers. *SIGCOMM Comput. Commun. Rev.* 42, 2 (March 2012), 80–86. https://doi.org/10.1145/2185376.2185387